

#42DAYSOFML CHALLENGE

Week 4

Revision Guide

Deep Learning · Days 25 – 31

Concepts · Code · Real World Problems · Interview Prep

WHAT'S INSIDE

DAY 24 Recap — Week 3 one-line recap before Week 4 begins

DAY 25 Pipelines

DAY 26 Neural Networks

DAY 27 CNNs

DAY 28 Transfer Learning

DAY 29 RNNs and LSTMs

DAY 30 Transformers

DAY 31 Backpropagation and Optimizers

DAY BONUS MNC Interview Q&A — Interview Questions · Code Syntax · Real World Examples

DAY 24 RECAP · BEFORE WE BEGIN

Week 3 built the full supervised and unsupervised ML toolkit. Here is the one-line summary of each day, so Week 4's move into Deep Learning builds on solid ground.

Day	Topic	One-Line Takeaway
17	Random Forests	A single tree overfits to noise. Bagging trains many trees on bootstrap samples and averages votes — variance drops, accuracy stabilises.
18	Overfitting vs Underfitting	100% train accuracy is a warning sign, not a success. Diagnose with a learning curve before reaching for regularization.
19	Model Evaluation	Accuracy lies on imbalanced data. Match the metric to the cost of being wrong — recall for missed cases, precision for false alarms.
20	Cross Validation & Tuning	One train/test split is noise. K-Fold CV gives a trustworthy number. Anything that learns from data belongs inside a Pipeline.
21	Gradient Boosting (XGBoost)	Bagging reduces variance in parallel; boosting reduces bias sequentially. Watch test loss per round — early stopping prevents overfit.
22	K-Means Clustering	First unsupervised algorithm in the series. Distance-based — always scale first. Elbow + silhouette score guide the choice of K.
23	Dimensionality Reduction (PCA)	Compresses correlated features into orthogonal components. 95% explained variance is the production threshold. Destroys interpretability.

WEEK 4 ROADMAP

Week 3 ended with PCA — the full classical ML toolkit. Week 4 moves into Deep Learning: Pipelines (the production glue that ties every preprocessing step to every model from Week 1-3), Neural Networks (the building block), CNNs (vision), Transfer Learning (why training from scratch is rarely necessary), RNNs and LSTMs (sequences), Transformers (the architecture behind every modern LLM), and Backpropagation and Optimizers (how all of it actually learns). By Day 31, you understand not just how to call `.fit()` on a deep model, but what's happening inside it.

SELF-CHECK BEFORE STARTING WEEK 4

Without notes: explain why a scaler fit on the full dataset before cross-validation causes data leakage. State what 95% explained variance means in PCA. Describe when you'd choose XGBoost over Random Forest. If any feel shaky, revisit that day before continuing.

DAY 25 · PIPELINES

THE CORE IDEA

Writing preprocessing and model training as separate steps looks fine until it breaks in production. A scaler fit on the full dataset before cross-validation leaks information from validation rows into training. A scaler saved separately from the model gets forgotten at deployment. sklearn's Pipeline chains every step — imputers, scalers, encoders, the model — into one object with one `.fit()` and one `.predict()`. `ColumnTransformer` extends this to route different column types (numerical, categorical, ordinal) through different sub-pipelines, then concatenates the results.

THE THREE SILENT FAILURE MODES PIPELINES PREVENT

Failure Mode	What Goes Wrong	How Pipeline Fixes It
Deployment bug	Engineer deploying the model forgets to apply the scaler to new data. Predictions are silently wrong — no error thrown.	One <code>joblib.dump()</code> saves scaler + model together. Loading the pipeline guarantees preprocessing runs.
CV leakage	Calling <code>scaler.fit_transform(X)</code> on the full dataset before CV means validation rows already influenced the scaler's mean/std.	Pipeline refits scaler on each training fold only — validation fold never seen during fit.
Train/serve skew	Training notebook and production code apply preprocessing in slightly different order or with different parameters.	Pipeline is one serialized object — identical transformation logic, identical order, every time.

WHAT `.FIT()` ACTUALLY DOES INSIDE A PIPELINE

```
pipe.fit(X_train, y_train)

Step 1: X_scaled = StandardScaler.fit_transform(X_train)
        -> scaler learns mean and std FROM TRAIN DATA ONLY

Step 2: LogisticRegression.fit(X_scaled, y_train)
        -> model trains on the scaled train data

pipe.predict(X_test)

Step 1: X_scaled = StandardScaler.transform(X_test)
        -> uses mean/std learned from TRAIN. Test stats never touch the scaler.

Step 2: LogisticRegression.predict(X_scaled)
```

FROM THE DAY 25 NOTEBOOK — CV LEAKAGE, MEASURED

100 samples, 50 features, only 5 informative, 10 redundant — a small, high-dimensional dataset built specifically to make the leakage gap visible. 5-fold StratifiedKFold, `KNeighborsClassifier`.

Leaky (scaler fit on all data before CV)	Correct (scaler refit inside Pipeline per fold)
Inflated CV score — model looks like it generalises when it has actually seen validation-set statistics during scaling.	Honest CV score — the only number that predicts how the model will behave on truly unseen data.

COLUMNTRANSFORMER — MIXED DATA TYPES

```

numerical_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])
categorical_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])
preprocessor = ColumnTransformer([
    ('num', numerical_pipeline, numerical_features),
    ('cat', categorical_pipeline, categorical_features),
])
full_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('clf', RandomForestClassifier(random_state=42))
])

```

GRIDSEARCHCV ON A PIPELINE — DOUBLE UNDERSCORE SYNTAX

Because the pipeline is one object, GridSearchCV can tune ANY step's hyperparameters using `stepname__parametername`. This means imputation strategy, model depth, and number of estimators get tuned in a single search, and CV is correct because the entire pipeline — including preprocessing — refits from scratch on every fold.

```

param_grid = {
    'preprocessor__num__imputer__strategy': ['mean', 'median'],
    'clf__n_estimators': [50, 100],
    'clf__max_depth': [3, 5, None]
}
grid_search = GridSearchCV(full_pipeline, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)

```

THE PRODUCTION TRAP — THE DEPLOYMENT SCALER BUG

A data scientist trains a model: fit a StandardScaler, fit a LogisticRegression on the scaled data. They save only the model with `joblib.dump(lr, 'model_only.pkl')` — the scaler object is never saved. Days later, an engineer loads the model for deployment. They don't know a scaler was applied during training. They call `lr.predict()` directly on raw, unscaled incoming data.

No error is thrown. The model returns predictions — just wrong ones. Every prediction in production is silently incorrect because the input distribution the model expects (scaled, mean=0, std=1) never matches what it's actually receiving.

Fix: `joblib.dump(pipe, 'pipeline.pkl')` saves scaler + model as one unit. Deployment code becomes three lines: load, predict, done. There's no scaler to forget because there's no separate scaler object at all.

GOLDEN RULE

If it has a `.fit()` method and learns from your data — scalers, imputers, feature selectors, encoders — it belongs inside the Pipeline. Every major ML platform (Vertex AI, SageMaker, Azure ML) expects models as pipeline objects for exactly this reason: it enforces that preprocessing is always applied, in the right order, with parameters learned only from training data.

DAY 26 · NEURAL NETWORKS

THE CORE IDEA

A neural network is a chain of mathematical operations applied to input data, layer by layer, until you get a prediction. Each layer computes $Z = X @ W + b$ (a weighted sum plus bias), then applies a non-linear activation $A = \text{activation}(Z)$. Without the activation function, stacking layers is pointless — multiple linear operations collapse into a single linear operation, no matter how many layers you stack. The non-linearity is what lets a network learn curves, not just straight lines.

ACTIVATION FUNCTIONS

Function	Formula	Range	Use Case
Sigmoid	$1 / (1 + e^{-x})$	(0, 1)	Binary classification output layer
ReLU	$\max(0, x)$	[0, infinity)	Hidden layers — most common choice by far
Tanh	$(e^x - e^{-x}) / (e^x + e^{-x})$	(-1, 1)	Hidden layers, RNNs

WHY NOT SIGMOID EVERYWHERE — VANISHING GRADIENTS

Sigmoid's maximum derivative is 0.25. During backpropagation, the chain rule multiplies gradients layer by layer. After just a few sigmoid layers, the gradient reaching early layers shrinks toward zero — those layers barely update. ReLU's derivative is exactly 1 for any positive input, so gradients pass through unchanged. This is the single biggest reason ReLU replaced sigmoid as the default for hidden layers.

FROM THE DAY 26 NOTEBOOK — GRADIENT MAGNITUDE AFTER 10 LAYERS

Layer	Sigmoid Gradient (x0.25 per layer)	ReLU Gradient (x1.0 per layer)
1	0.25000000	1.0
5	0.00097656	1.0
10	0.0000009537 — essentially zero	1.0 — unchanged

After 10 sigmoid layers, the gradient is roughly 1 in a billion. That layer's weights effectively stop learning. ReLU's gradient stays exactly 1.0 through all 10 layers, for any neuron that's active.

FORWARD PASS, LOSS, AND BACKPROPAGATION — THE TRAINING LOOP

Every training iteration runs three steps: (1) forward pass — compute predictions by pushing X through every layer; (2) compute loss — Binary Cross-Entropy for binary classification, $L = -\text{mean}(y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y}))$; (3) backward pass — the chain rule computes how much each weight contributed to the loss, then nudges every weight in the direction that reduces it: $W = W - \text{learning_rate} * dL/dW$.

KEY SYNTAX — BUILDING THE SAME NETWORK IN KERAS (MNIST)

```
model = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=(784,)),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dense(10, activation='softmax') # 10 digit classes
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
history = model.fit(X_train_flat, y_train, epochs=10,
                   batch_size=128, validation_split=0.1)
test_loss, test_acc = model.evaluate(X_test_flat, y_test)
```

THE PRODUCTION TRAP — VANISHING GRADIENTS AT SCALE

A startup builds a 10-layer neural network to classify customer support tickets into 50 categories. They use sigmoid activations throughout every hidden layer. After 100 epochs, training accuracy is stuck at 42%.

The first 7 layers are barely learning. Their gradients are roughly 0.0001 by the time backpropagation reaches them — sigmoid's max derivative of 0.25, multiplied across 7 layers, is $0.25^7 = 0.00006$. Effectively zero.

Fix: replace sigmoid with ReLU in every hidden layer (keep sigmoid or softmax only on the output layer where bounded probabilities are needed). Add BatchNorm between layers for additional training stability. Training accuracy recovers within 15-20 epochs.

GOLDEN RULE

ReLU is the default for hidden layers in virtually every modern architecture, not because it's mathematically elegant, but because it keeps gradients flowing through deep networks. Reserve sigmoid for binary output layers and softmax for multi-class output layers — never stack them in hidden layers of a deep network.

DAY 27 · CNNs

THE CORE IDEA

A fully connected layer treats every pixel as an independent input with no awareness that nearby pixels are spatially related. For a 224x224 RGB image, that's 150,528 inputs, and the network would need to relearn what an edge looks like separately for every possible position. CNNs fix this with convolution: a small filter slides across the image, computing dot products at each position. The same filter scans the entire image — weight sharing — so a vertical-edge detector learned at one position works everywhere.

THE THREE KEY OPERATIONS

Operation	What It Does
Convolution	Slide a learnable filter (e.g. 3x3) over the input. One filter produces one feature map. Output size: $(\text{input} + 2 \times \text{padding} - \text{kernel}) / \text{stride} + 1$.
Pooling	Downsample the feature map. Max pooling keeps the strongest activation in each region — reduces spatial size, gives translation invariance.
Activation (ReLU)	Adds non-linearity after every conv layer — same reason as Day 26.

HOW CNN LAYERS BUILD UP UNDERSTANDING

Layer 1 detects edges and colour gradients. Layer 2 combines edges into corners and textures. Layer 3+ combines textures into object parts — wheels, eyes, ears. Final layers combine parts into full objects. This hierarchy was directly visualised in Zeiler and Fergus's 2014 paper on AlexNet filters — it isn't a theory, it's observable in the trained weights.

FROM THE DAY 27 NOTEBOOK — STRIDE AND PADDING ON OUTPUT SIZE

Formula: $(\text{input_size} + 2 \times \text{padding} - \text{kernel_size}) // \text{stride} + 1$

Configuration	8x8 Input -> Output
No padding, stride 1 (default)	8x8 -> 6x6
Same padding (pad=1), stride 1	8x8 -> 8x8
No padding, stride 2	8x8 -> 3x3
Same padding (pad=1), stride 2	8x8 -> 4x4

KEY SYNTAX — A 3-BLOCK CNN ON CIFAR-10 (PYTORCH)

```

class CNNClassifier(nn.Module):
    def __init__(self):
        super().__init__()
        self.block1 = nn.Sequential(
            nn.Conv2d(3, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.Conv2d(32, 32, 3, padding=1), nn.BatchNorm2d(32), nn.ReLU(),
            nn.MaxPool2d(2, 2), nn.Dropout2d(0.25)
        )
        # block2: 32->64 channels, block3: 64->128 channels (same pattern)
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Linear(128*4*4, 512), nn.ReLU(),
            nn.Dropout(0.5), nn.Linear(512, 10)
        )
    def forward(self, x):
        return self.classifier(self.block3(self.block2(self.block1(x))))

```

FROM THE DAY 27 NOTEBOOK — A REAL TRAINING RUN

10 epochs on CIFAR-10 (50,000 train / 10,000 test images, 10 classes). A 3-block CNN with BatchNorm and Dropout reaches roughly 75-85% test accuracy — against a 10% random baseline. Per-class accuracy reveals the model isn't uniform: vehicle classes (plane, car, truck, ship) typically score highest. Cat and deer are the most commonly confused pair — they share similar shape and texture at 32x32 resolution.

Visualising the first conv layer's 32 filters after training shows clear edge detectors and colour-gradient detectors — exactly the hierarchy the concept section describes, visible in the actual learned weights.

THE PRODUCTION TRAP — ACCURACY VS INFERENCE SPEED

Google Photos classifies roughly 1.2 billion photos per month. At 100ms per image, that's about 138 days of compute monthly. At 10ms per image, 13.8 days. The production question is never 'build the most accurate CNN' — it's 'find the accuracy/speed tradeoff that meets the SLA.'

What engineers actually do: depthwise separable convolutions (MobileNet) — 8-9x fewer FLOPs for a 1-2% accuracy drop. Quantisation — int8 weights, 4x smaller model, 2-4x faster, under 1% accuracy drop. Pruning — remove low-magnitude weights; ResNet-50 prunes to 40% of its size for roughly 2% accuracy drop.

Decision rule: server/batch inference optimises for accuracy (ResNet, EfficientNet). Mobile/edge optimises for size and speed (MobileNetV3). Real-time video optimises for latency (YOLO-family).

GOLDEN RULE

Weight sharing and local connectivity are what make CNNs efficient for images — the same 3x3 filter scanning the whole image needs orders of magnitude fewer parameters than a fully connected layer would. BatchNorm after every conv layer is now close to mandatory: it stabilises training and lets you use higher learning rates without divergence.

DAY 28 · TRANSFER LEARNING

THE CORE IDEA

Training a CNN from scratch needs millions of labelled images and days of GPU time — most real tasks don't have that. A model already trained on 1.2 million ImageNet images has learned to detect edges, textures, shapes, and object parts. Those low-level features transfer to almost any visual task. Transfer learning reuses that knowledge instead of relearning it from random weights.

TWO STRATEGIES

Strategy	When And Why
Feature Extraction	Freeze ALL pretrained layers. Only train the final classification head. Fast. Best when your dataset is small (under ~1,000 images) and close to ImageNet's domain.
Fine-tuning	Unfreeze some or all pretrained layers, train them at a very low learning rate alongside the new head. Slower. Better with more data, or when your domain differs from ImageNet (medical, satellite imagery).

FROM THE DAY 28 NOTEBOOK — SAME DATA, THREE STRATEGIES

Intentionally small dataset: 5 classes from CIFAR-10, only 200 training images per class (1,000 total) — built to make the gap visible, the same way a real low-data production scenario looks.

Approach	Trainable Params	Result Pattern
From scratch (SimpleCNN)	100% of network	Weakest — overfits fast on 1,000 images
Feature extraction (MobileNetV2, frozen)	~1% of network (head only)	Strong, fast to train
Fine-tuning (last 3 blocks unfrozen)	~15-20% of network	Best — highest final accuracy

Same architecture, same data, same epochs. The only variable is the starting point. Fine-tuning's two-phase approach — train the head first at $lr=0.001$, then unfreeze the last blocks at $lr=1e-5$ — consistently outperforms training from scratch by a wide margin on small datasets.

KEY SYNTAX — FEATURE EXTRACTION VS FINE-TUNING

```
# Feature extraction: freeze everything, replace the head
model = models.mobilenet_v2(pretrained=True)
for param in model.parameters():
    param.requires_grad = False
model.classifier = nn.Sequential(nn.Dropout(0.2), nn.Linear(1280, num_classes))

# Fine-tuning: unfreeze last 3 blocks, use differential learning rates
for i in range(len(model.features) - 3, len(model.features)):
    for param in model.features[i].parameters():
        param.requires_grad = True
optimizer = optim.Adam([
    {'params': model.features.parameters(), 'lr': 1e-5}, # pretrained: low LR
    {'params': model.classifier.parameters(), 'lr': 1e-4} # new head: higher LR
])
```

WHY THE SAME NORMALISATION MATTERS

ImageNet-pretrained weights were calibrated against ImageNet's specific input statistics: mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]. Feed in differently normalised inputs and the pretrained layers' activations land on a completely different scale than what they were designed for — they stop behaving as intended.

THE PRODUCTION TRAP — DOMAIN SHIFT

A startup fine-tunes ResNet50 on lab-condition plant disease photos and reaches 91% validation accuracy. In production, performance drops to roughly 67%.

Their validation images were uniform lighting, clean backgrounds, controlled lab setting. Production photos come from farmers in variable lighting, different angles, cluttered backgrounds — a different distribution than what the model was fine-tuned on.

Fix: domain-specific augmentation during training (lighting jitter, rotation, background variation matching real conditions), test-time augmentation at inference, and monitoring input image statistics (brightness, contrast) in production as an early warning signal for distribution shift.

GOLDEN RULE

Feature extraction when your dataset is small and close to ImageNet's domain. Fine-tuning when you have more data or your domain is different. Always use differential learning rates when fine-tuning — 10x lower for pretrained layers than for the new head — or you risk catastrophic forgetting overwriting the pretrained representations with noise.

DAY 29 · RNNs AND LSTMs

THE CORE IDEA

A feedforward network has no memory of previous inputs — fine for classifying one image, useless for predicting the next word in a sentence where meaning depends on what came before. RNNs add a hidden state passed from one time step to the next: $h_t = \tanh(W_h * h_{t-1} + W_x * x_t + b)$. The hidden state is the network's memory of everything it has seen so far in the sequence.

THE VANISHING GRADIENT PROBLEM

Backpropagation through time (BPTT) must flow gradients back through every time step. Each step multiplies the gradient by the weight matrix's values — typically less than 1 after tanh. After 20-30 steps, the gradient reaching early time steps is essentially zero. The network cannot learn dependencies that span more than a couple dozen steps.

FROM THE DAY 29 NOTEBOOK — GRADIENT MAGNITUDE OVER 50 STEPS

Simulated gradient flow with $W=0.9$, $\tanh'=0.5$ (a typical, not even worst-case, configuration):

Position	Gradient Magnitude
Step 49 (final step)	~0.5 (full signal)
Step 40 (10 steps back)	Single-digit percent of original
Step 0 (50 steps back)	~1e-50 — effectively zero

The network literally cannot learn anything that happened more than roughly 20 steps ago in a vanilla RNN. This is the exact problem LSTMs were built to solve.

HOW LSTM GATES FIX IT — ADDITIVE, NOT MULTIPLICATIVE

LSTMs add a cell state — a separate memory channel that runs through the sequence with only additive updates, avoiding the repeated multiplication that causes vanishing gradients. Three gates control it:

Gate	What It Controls
Forget gate (f_t)	Which information from the previous cell state to erase. Output in [0,1] — 0 forgets completely, 1 keeps completely.
Input gate (i_t)	Which new information to write into the cell state.
Output gate (o_t)	What to read from the cell state as the hidden state for this time step.

Cell state update: $c_t = f_t * c_{prev} + i_t * \hat{c}_t$ — additive, not multiplied repeatedly across time steps. This is the entire mechanism that lets gradients flow back thousands of steps without vanishing.

KEY SYNTAX — LSTM LANGUAGE MODEL AND BIDIRECTIONAL SENTIMENT

```
self.lstm = nn.LSTM(input_size=embed_dim, hidden_size=hidden_dim,
                    num_layers=2, dropout=0.3, batch_first=True)
lstm_out, (h_n, c_n) = self.lstm(embedded)

# Bidirectional — reads sequence forward AND backward
self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True, bidirectional=True)
# Always clip gradients for RNN/LSTM training:
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

FROM THE DAY 29 NOTEBOOK — GENERATION AND CLASSIFICATION

A 2-layer LSTM language model trained on an ML-concepts corpus learns to generate plausible continuations, e.g. seeding with 'machine learning is a subset of' produces coherent next words. Perplexity (exp of cross-entropy loss) drops steadily over 50 epochs with gradient clipping at `max_norm=1.0`.

A bidirectional LSTM sentiment classifier on a small ML-domain dataset (positive: good convergence, high accuracy; negative: overfitting, failed training) correctly classifies new sentences with high confidence — e.g. flags 'training failed and loss did not converge' as negative with strong confidence.

THE PRODUCTION TRAP — VARIABLE SEQUENCE LENGTH AT INFERENCE

An LSTM-based fraud detector is trained on sequences of exactly 50 events. In production, some transactions arrive with 200 events in history, others with just 3. The model throws a runtime error roughly 2% of the time — exactly on the high-value transactions with long histories.

Fix: use `nn.utils.rnn.pack_padded_sequence` so the LSTM skips padding instead of processing it. Store the exact preprocessing (truncation/padding length) as a versioned artifact alongside the model, not a hardcoded constant someone can silently change. Add an explicit assertion at the top of the inference function checking input shape.

GOLDEN RULE

The forget gate bias is initialised to 1, not 0 — this means the LSTM starts by defaulting to 'remember everything,' which keeps gradients flowing through the cell state from step one of training. In 2024+, LSTMs are still the right choice for time series, on-device/edge inference, and real-time streaming where Transformer attention's $O(n^2)$ cost is too expensive.

DAY 30 · TRANSFORMERS

THE CORE IDEA

The 2017 paper 'Attention Is All You Need' (Vaswani et al., Google Brain) replaced recurrence entirely with self-attention: every token attends to every other token simultaneously, in parallel, with no sequential bottleneck. This solved RNNs' two core limits — sequential computation that blocks parallelism, and long-range dependencies that require surviving 100+ recurrent steps.

THE THREE MATRICES — Q, K, V

Matrix	Role
Query (Q)	What am I looking for?
Key (K)	What do I have to offer?
Value (V)	What information do I carry?

Attention(Q, K, V) = $\text{softmax}(QK^T / \sqrt{d_k}) * V$ — the dot product QK^T measures compatibility between every query and every key; dividing by $\sqrt{d_k}$ prevents softmax saturation from large values; the result weights a sum over the values.

MULTI-HEAD ATTENTION AND POSITIONAL ENCODING

Multi-head attention runs self-attention h times in parallel with different learned projections — one head might learn syntactic structure, another semantic similarity, another coreference. Self-attention itself is permutation-invariant (it has no sense of word order), so positional encodings — sinusoidal functions added to token embeddings — inject that order back in.

FROM THE DAY 30 NOTEBOOK — ATTENTION WEIGHTS, MEASURED

Self-attention implemented from the exact paper equation on the sentence 'the cat sat on the mat.' For the query token 'cat', attention weights across all 6 tokens sum to exactly 1.0 — a verified probability distribution, not an approximation. Scaled dot-product attention, multi-head split into 8 heads of $d_k=8$ each ($d_{\text{model}}=64$), and a full Transformer encoder block (LayerNorm -> MultiHeadAttention -> residual -> LayerNorm -> FFN -> residual) were all implemented and shape-verified.

Component	Verified Property
Self-attention (6-token sentence)	Attention weights for each query sum to 1.0 across all keys
Positional encoding (sinusoidal)	Nearby positions show higher cosine similarity than distant ones
Multi-head attention (8 heads)	Output shape matches input shape; attention weights sum to 1.0 per head
Mini Transformer encoder (2 layers)	Far under 1% of BERT-base's 110M parameters — same architecture, smaller scale

KEY SYNTAX — BERT IN 15 LINES (HUGGINGFACE)

```
from transformers import pipeline
sentiment_pipeline = pipeline(
    task='text-classification',
    model='distilbert-base-uncased-finetuned-sst-2-english'
)
results = sentiment_pipeline([
    'This model achieves state of the art results.',
    'Training diverged and loss never decreased.'
])
# [{'label': 'POSITIVE', 'score': 0.99}, {'label': 'NEGATIVE', 'score': 0.99}]
```

BERT's WordPiece tokenizer adds [CLS] at the start (its final embedding drives classification) and [SEP] at the end. Long or uncommon words split into subwords — 'backpropagation' and 'hyperparameter' typically split into multiple WordPiece tokens, not one. Visualising attention heads on a real trained DistilBERT shows different heads learning different patterns: some attend to adjacent tokens (local syntax), some to [CLS]/[SEP], some capture longer-range relationships — consistent with Clark et al.'s 2019 'What Does BERT Look At?'

THE PRODUCTION TRAP — CONTEXT WINDOW LIMITS

A legal tech company fine-tunes BERT (512-token hard limit) for contract clause classification. Most contracts run 5,000-50,000 tokens. Their solution: truncate to the first 512 tokens.

High-value contracts get misclassified at twice the rate of short ones — important clauses (indemnification, limitation of liability) are often near the end, outside the 512-token window entirely.

What production teams actually do: sliding window with overlap (chunk into 512-token windows, aggregate predictions), hierarchical models (sentence-level then document-level), Longformer/BigBird for sparse attention up to 16,384 tokens, or two-stage retrieval (TF-IDF/BM25 finds relevant sections, BERT runs only on those).

GOLDEN RULE

Encoder-only (BERT) for understanding tasks — classification, NER, QA. Decoder-only (GPT, LLaMA) for generation. Encoder-decoder (T5, BART) for transforming input to output — translation, summarisation. Self-attention's $O(n^2)$ cost in sequence length is the reason long-context models need sparse or linear attention approximations rather than naive scaling.

DAY 31 · BACKPROPAGATION AND OPTIMIZERS

THE CORE IDEA

Backpropagation computes dL/dW for every weight using the chain rule, working backward from the loss through each layer, reusing intermediate results along the way. Once you have gradients, an optimizer decides how to use them to update weights. The simplest rule, SGD, is $W = W - lr * dL/dW$ — but it oscillates in narrow loss valleys, slows in flat regions, and uses one learning rate for every parameter regardless of how that parameter's gradient actually behaves.

FROM THE DAY 31 NOTEBOOK — MANUAL BACKPROP VERIFIED AGAINST PYTORCH

A 2-layer network (Linear → ReLU → Linear → Sigmoid) was implemented fully from scratch with NumPy — forward pass, Binary Cross-Entropy loss, and backward pass via explicit chain rule. Every gradient ($dW1$, $db1$, $dW2$, $db2$) was then compared against PyTorch autograd on identical weights and identical data.

Gradient	Manual vs Autograd	Result
$dW1$, $db1$, $dW2$, $db2$	Max diff < $1e-5$ on all four	MATCH

Every gradient matched PyTorch's autograd to within numerical precision — confirming the chain-rule implementation is mathematically correct, not just plausible-looking.

SGD VS MOMENTUM VS RMSPROP VS ADAM

Optimizer	What It Fixes
SGD	$W = W - lr * grad$. Simple, but oscillates in narrow valleys and uses one LR for every parameter.
Momentum	Adds a velocity term: $v = beta*v - lr*grad$. Past gradients influence the current update — smooths oscillation.
RMSProp	Adapts LR per parameter using a running average of squared gradients: large gradients → smaller steps, small gradients → larger steps.
Adam	Combines Momentum (1st moment) + RMSProp (2nd moment), plus bias correction for early training. Default optimizer for almost every network today.

FROM THE DAY 31 NOTEBOOK — ALL FOUR, SAME NETWORK, SAME DATA

All four optimizers built from scratch in NumPy and run on the identical 2-layer network against the make_moons dataset, same random seed, same 200 epochs:

Optimizer	Convergence Pattern Observed
SGD ($lr=0.05$)	Slowest, often visibly noisier loss curve and the most jagged decision boundary
SGD + Momentum ($beta=0.9$)	Noticeably smoother and faster than plain SGD on the same LR
RMSProp ($lr=0.001$)	Fast early progress, adapts step size automatically per parameter
Adam ($lr=0.001$)	Fastest and most stable convergence of the four — smoothest decision boundary by epoch 200

WHY ADAM'S BIAS CORRECTION MATTERS

Adam's moment estimates m and v start at zero. Without bias correction, the first several steps' effective step size is 10-100x smaller than the target learning rate — the network barely moves in early training.

With correction ($m_hat = m/(1-\beta_1^t)$, $v_hat = v/(1-\beta_2^t)$), step sizes reach near the target LR from step 1. This is why production Adam implementations always include bias correction by default.

KEY SYNTAX — LEARNING RATE SCHEDULERS

```
# Warmup + cosine — standard for Transformer training (BERT, GPT, T5, LLaMA)
def lr_lambda(step):
    if step < warmup_steps:
        return step / warmup_steps          # linear warmup
    progress = (step - warmup_steps) / (total_steps - warmup_steps)
    return 0.5 * (1 + math.cos(math.pi * progress))  # cosine decay
scheduler = optim.lr_scheduler.LambdaLR(optimizer, lr_lambda)

# Gradient clipping — call after loss.backward(), before optimizer.step()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
```

From the Day 31 notebook's learning-rate sweep on the same network: $lr=0.1$ diverges (steps overshoot the minimum), $lr=0.01$ converges fast, $lr=0.001$ is the reliable Adam default, $lr=0.0001$ converges slowly but works, and $lr=0.00001$ barely moves in 100 epochs. The 5-curve comparison makes the divergence/underfitting boundary visible directly in the loss plot.

THE PRODUCTION TRAP — LEARNING RATE BREAKING A TRAINING RUN

A team trains a large image classifier on a cloud GPU. Loss drops for 2 hours, then spikes and never recovers. They restart with identical settings — it happens again. Cost: \$800 in wasted compute.

Root cause: they increased batch size from 32 to 256 for GPU efficiency but didn't scale the learning rate. Larger batches give lower-variance (more 'confident') gradient estimates, so the same LR produces a larger effective step — large enough to overshoot and destabilise training.

Fix — the linear scaling rule (Goyal et al., 2017): multiplying batch size by k should multiply learning rate by k , combined with linear warmup over the first several epochs. Two other common LR bugs: not resetting optimizer state when resuming from a checkpoint, and using the same LR for a pretrained backbone and a new head during fine-tuning (should be 10-100x lower for the backbone).

GOLDEN RULE

Adam (or AdamW for Transformers) is the safe default optimizer for almost everything in 2024+. Always pair RNN/LSTM training with gradient clipping ($max_norm=1.0$ for Transformers, 5.0 for RNNs is a common starting point). When increasing batch size, scale the learning rate proportionally and add warmup — this single rule prevents one of the most common and expensive production training failures.

WEEK 4 · ALL INTERVIEW QUESTIONS AT A GLANCE · MNC INTERVIEW Q&A

Pipelines · Neural Networks · CNNs · Transfer Learning · RNNs/LSTMs · Transformers · Backprop & Optimizers

DAY 25 · Q1 · GENERAL MLE ROUND

Your scaler is fit on the full dataset before cross-validation starts. What went wrong and how do you fix it?

The scaler computes mean and std from the full dataset, including every row that will later appear as validation data in CV. When the model is evaluated on those rows, they've already influenced the feature scaling — the CV score is inflated because the model benefited from statistical information about the validation set during training.

Fix: wrap the scaler inside a sklearn Pipeline and pass the Pipeline to `cross_val_score` or `GridSearchCV`. The Pipeline refits the scaler on each training fold separately, so validation data never touches preprocessing until it's time to score.

DAY 25 · Q2 · AMAZON — ML ENGINEER ROUND

How do you tune hyperparameters of a preprocessing step and a model together without introducing leakage?

Build a Pipeline containing both the preprocessing steps (imputer, scaler, encoder) and the model as named steps. Pass that Pipeline — not pre-transformed data — directly to `GridSearchCV`.

Reference each step's hyperparameter using double-underscore syntax: `'clf__n_estimators'`, `'preprocessor__num__imputer__strategy'`. `GridSearchCV` refits the entire Pipeline on every fold for every parameter combination, so preprocessing is always learned only from that fold's training data.

DAY 26 · Q1 · GENERAL MLE ROUND

Why does stacking linear layers without activation functions accomplish nothing?

A linear transformation followed by another linear transformation is still just a linear transformation — mathematically, $Z2 = W2(W1 \cdot X + b1) + b2 = (W2 \cdot W1) \cdot X + (W2 \cdot b1 + b2)$, which is equivalent to a single linear layer with combined weights.

Without a non-linear activation between layers, a 100-layer network has the exact same representational power as a 1-layer network. The activation function is what gives depth any meaning — it's what lets the network learn curves and non-linear decision boundaries instead of only straight lines.

DAY 26 · Q2 · FLIPKART — ML ENGINEER ROUND

Your 10-layer network with sigmoid activations throughout is stuck at 42% training accuracy after 100 epochs. Diagnose it.

Vanishing gradients. Sigmoid's maximum derivative is 0.25. Backpropagation multiplies gradients across layers via the chain rule — after 7 layers, 0.25^7 is roughly 0.00006, essentially zero by the time the signal reaches early layers. Those layers stop updating meaningfully.

Fix: replace sigmoid with ReLU in every hidden layer (sigmoid's derivative of exactly 1 for positive inputs doesn't shrink). Keep sigmoid or softmax only on the output layer where bounded probabilities are actually needed. Add BatchNorm between layers for further stability.

DAY 27 · Q1 · GOOGLE — ML ENGINEER ROUND

Why are CNNs better than fully connected networks for image data?

Three reasons. Weight sharing: the same filter detects an edge anywhere in the image, instead of needing separate weights for every pixel position. Local connectivity: a 3x3 conv only looks at a 3x3 region, matching images' local structure where nearby pixels are correlated. Translation invariance: max pooling makes the representation robust to small shifts.

Together these reduce parameter count by orders of magnitude — ResNet-50 has roughly 25M parameters; a fully-connected equivalent at the same input size would need billions — while encoding the right inductive bias for spatial data.

DAY 27 · Q2 · SWIGGY — ML ENGINEER ROUND

A CNN deployed in production gives different results on the same image at different times. What's the likely cause?

Most likely the model is in training mode (`model.train()`) instead of eval mode (`model.eval()`) at inference time. Dropout randomly zeros activations during training, and BatchNorm uses batch statistics instead of running statistics — both non-deterministic.

Fix: always call `model.eval()` before inference. Also check GPU non-determinism (set `torch.backends.cudnn.deterministic = True` if needed) and confirm the preprocessing pipeline has no random augmentations applied at inference time.

DAY 28 · Q1 · GENERAL MLE ROUND

When would you choose feature extraction over fine-tuning, and vice versa?

Feature extraction when the dataset is very small (under roughly 1,000 images) — fine-tuning would overfit the pretrained layers on too little data — and when the task domain is close to ImageNet's natural-image domain.

Fine-tuning when you have more data (5,000+ images) or your domain differs meaningfully from ImageNet (medical imaging, satellite imagery), and when you need the highest possible accuracy and can afford the extra training time and differential learning rates.

DAY 28 · Q2 · META — ML ENGINEER ROUND

You fine-tune a pretrained model and it performs worse than the frozen pretrained baseline. What happened?

Catastrophic forgetting — fine-tuning aggressively overwrote the pretrained representations with noise. Usually caused by a learning rate that's too high for the pretrained layers, too many layers unfrozen on too little data, or training for too many epochs.

Fix: use differential learning rates (roughly 10x lower for pretrained layers than for the new head), train only the head first for a few epochs before unfreezing anything, and use early stopping on validation loss.

DAY 29 · Q1 · GENERAL MLE ROUND

Explain the vanishing gradient problem in RNNs and how LSTMs solve it.

In vanilla RNNs, backpropagation through time multiplies the gradient by the weight matrix at every time step. If the matrix's spectral radius is under 1, gradients shrink exponentially with sequence length — after 20-30 steps the signal at early time steps is near zero and those weights can't update based on long-range dependencies.

LSTMs solve this with the cell state: a separate memory channel using additive updates ($c_t = f_t * c_{prev} + i_t * \hat{c}$) instead of repeated multiplication. The gradient flows through this 'highway' without being repeatedly multiplied toward zero.

DAY 29 · Q2 · AMAZON — ML ENGINEER ROUND

In 2024, when would you still choose an LSTM over a Transformer for a production task?

Time series forecasting on tabular sequential data (stock prices, sensor readings) — LSTMs often match Transformer quality at much lower compute. On-device or edge inference where model size is tightly constrained — a small LSTM is tiny compared to even a small Transformer.

Real-time streaming where you need to process one token at a time without recomputing attention over the entire history (LSTM state update is $O(1)$ per step versus Transformer attention's $O(n^2)$), and applications with very long sequences (10k+ steps) where full attention is computationally infeasible.

DAY 30 · Q1 · GOOGLE — ML ENGINEER ROUND***Why does self-attention scale as $O(n^2)$ with sequence length, and what are the implications?***

Every token attends to every other token. For a sequence of n tokens, the attention matrix is $n \times n$ — computing QK^T is $O(n^2 * d_k)$ in time and $O(n^2)$ in memory to store the attention matrix. For $n=512$ (BERT) this is manageable; for $n=16,384$ the attention matrix alone is roughly 1GB per layer at float32.

This is why long-context Transformers use sparse attention patterns (Longformer's sliding window plus global attention) or linear approximations (Performer's kernel trick) to reduce to $O(n \log n)$ or $O(n)$ instead of scaling naively.

DAY 30 · Q2 · MICROSOFT — ML ENGINEER ROUND***What's the difference between encoder-only, decoder-only, and encoder-decoder Transformer architectures?***

Encoder-only (BERT): bidirectional self-attention, sees the full input at once. Used for understanding tasks — classification, NER, question answering. Decoder-only (GPT-4, LLaMA): causal self-attention, each token only attends to previous tokens. Used for generation — text generation, code completion, chat.

Encoder-decoder (T5, BART): encoder processes the input, decoder generates the output. Used for seq2seq tasks — translation, summarisation. Choice depends on the task: understand input -> encoder-only; generate from scratch -> decoder-only; transform input to output -> encoder-decoder.

DAY 31 · Q1 · FLIPKART — ML ENGINEER ROUND***Your model trains well with batch size 32 but diverges with batch size 512. Why?***

Larger batches give lower-variance gradient estimates, so each update is more 'confident' and takes a larger effective step in parameter space with the same learning rate. This can overshoot the loss minimum and destabilise training.

Fix: scale the learning rate linearly with batch size (the linear scaling rule, Goyal et al. 2017) — going from 32 to 512 (16x) should increase LR by roughly 16x, paired with a linear warmup over the first several epochs. Alternative: use gradient accumulation to simulate large batches without changing the LR at all.

DAY 31 · Q2 · GENERAL MLE ROUND***What is gradient clipping, when should you use it, and what value should you clip to?***

Gradient clipping caps the L2 norm of the gradient vector — if the norm exceeds a threshold, all gradients are scaled down proportionally before the weight update. This prevents one large gradient spike from throwing training into a bad region.

Use it for RNN/LSTM training (exploding gradients are common), Transformer training (occasional gradient spikes), and any task showing sudden loss spikes. Common values: `max_norm=1.0` for Transformers (BERT, GPT default), `max_norm=5.0` as a common RNN starting point. Always call it after `loss.backward()` and before `optimizer.step()`.

WEEK 4 — ALL QUESTIONS AT A GLANCE

Day · Company	Question	Key Points
25 · General	Scaler fit on full dataset before CV — what's wrong?	Leakage. Wrap scaler in Pipeline so it refits on each training fold only.
25 · Amazon	Tune preprocessing + model together without leakage?	Pipeline with double-underscore param names, passed whole to GridSearchCV.
26 · General	Why does stacking linear layers without activations fail?	Collapses into one linear layer. Activations are what give depth meaning.
26 · Flipkart	10-layer sigmoid network stuck at 42% accuracy?	Vanishing gradients. Swap to ReLU in hidden layers, add BatchNorm.
27 · Google	Why are CNNs better than FC networks for images?	Weight sharing + local connectivity + translation invariance via pooling.
27 · Swiggy	Same image, different predictions at different times?	Model in train() mode, not eval(). Dropout/BatchNorm non-deterministic.
28 · General	Feature extraction vs fine-tuning — when each?	Extraction: small data, close domain. Fine-tune: more data, different domain.
28 · Meta	Fine-tuned model worse than frozen baseline?	Catastrophic forgetting. Use differential LR, unfreeze gradually.
29 · General	Vanishing gradients in RNNs — how do LSTMs fix it?	Cell state uses additive updates instead of repeated multiplication.
29 · Amazon	When still choose LSTM over Transformer in 2024?	Time series, edge inference, real-time streaming, very long sequences.
30 · Google	Why does self-attention scale $O(n^2)$?	Every token attends to every token. Sparse/linear attention fixes long context.
30 · Microsoft	Encoder-only vs decoder-only vs encoder-decoder?	BERT=understand. GPT=generate. T5=transform input to output.
31 · Flipkart	Trains fine at batch=32, diverges at batch=512?	Scale LR linearly with batch size, add warmup, or use grad accumulation.
31 · General	What is gradient clipping and when to use it?	Caps gradient L2 norm. Use for RNNs/Transformers. max_norm=1.0 typical.

SELF-TEST BEFORE WEEK 5

Without notes: explain why an unfrozen pretrained backbone needs a lower learning rate than a freshly initialised head. State what problem the cell state's additive update solves in an LSTM. Describe why self-attention needs positional encoding when recurrence doesn't. Name two things that must live inside a Pipeline before cross-validation runs. If any feel shaky, go back to that day.

Week 5 starts with NLP and LLMs: Tokenization and Embeddings -> RAG -> Agents -> Vector Databases

Full notebooks: github.com/VaishnaviJagtap18/42-Days-of-ML-Challenge · #42DaysOfML